

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Курсовой проект
по курсу «Операционные системы»**

Выполнил: П. Д. Косарим
Группа: М8О-207БВ-24
Преподаватель: Е. С. Миронов

Москва, 2025

Условие

Цель работы:

1. Приобретение практических навыков в использовании знаний, полученных в течении курса
2. Проведение исследования в выбранной предметной области

Задание: Необходимо спроектировать и реализовать программный прототип в соответствии с выбранным вариантом. Произвести анализ и сделать вывод на основании данных, полученных при работе программного прототипа.

Консоль-серверная игра. Необходимо написать консоль-серверную игру. Необходимо написать 2 программы: сервер и клиент. Сначала запускается сервер, а далее клиенты соединяются с сервером. Сервер координирует клиентов между собой. При запуске клиента игрок может выбрать одно из следующих действий (возможно больше, если предусмотрено вариантом):

- Создать игру, введя ее имя
- Присоединиться к одной из существующих игр по имени игры «Быки и коровы» (угадывать необходимо числа). Общение между сервером и клиентом необходимо организовать при помощи `memory map`. При создании каждой игры необходимо указывать количество игроков, которые будут участвовать. То есть угадывать могут несколько игроков. Должна быть реализована функция поиска игры, то есть игрок пытается войти в игру не по имени, а просто просит сервер найти ему игру.

Вариант: 12

Метод решения

1. Создаем разделяемую память для данных сервера и канала обмена запросами/ответами.
2. Инициализируем сервер: создаем мьютекс для синхронизации и два семафора для уведомлений о запросах и ответах.
3. Запускаем серверный процесс, который в бесконечном цикле:
 - Ожидает запросы от клиентов через семафор
 - Обрабатывает запросы (создание игры, присоединение, ходы игроков)
 - Отправляет ответы клиентам через семафор
 - Управляет состоянием игр (активация, удаление завершенных, проверка активности игроков)
4. Клиентский процесс подключается к существующей разделяемой памяти и:
 - Предлагает меню действий: создать игру, присоединиться по имени, найти игру автоматически
 - Отправляет запросы в разделяемую память и ожидает ответа от сервера
 - В режиме игры периодически запрашивает состояние и делает ходы при своей очереди

Описание программы

`server.c` - Серверный процесс:

1. Создает и настраивает разделяемую память для данных сервера и канала запросов
2. Инициализирует синхронизационные примитивы: мьютекс и два семафора
3. Обрабатывает сигналы завершения для корректного завершения работы
4. В основном цикле ожидает запросы от клиентов через семафор

5. Обрабатывает запросы: создание игры, присоединение к игре, поиск игры, выполнение ходов, получение состояния
6. Управляет жизненным циклом игр: активирует при полном наборе игроков, удаляет старые завершённые игры
7. Отслеживает активность игроков и помечает неактивных
8. Отправляет ответы клиентам через семафор
9. При завершении освобождает все ресурсы (память, семафоры, мьютекс) и корректно завершает работу

client.c - Клиентский процесс:

1. Запрашивает у пользователя имя игрока и подключается к запущенному серверу
2. Открывает существующую разделяемую память сервера и канала запросов
3. Подключается к синхронизационным примитивам сервера
4. В режиме меню предлагает выбор действий: создать игру, присоединиться по имени, найти игру автоматически, показать все игры
5. Отправляет запросы серверу через разделяемую память и ожидает ответов с таймаутом
6. В режиме игры периодически запрашивает состояние игры у сервера
7. Отображает текущее состояние: игроков, чей ход, последние ходы
8. При своей очереди хода запрашивает у пользователя предположение (4 уникальные цифры)
9. Отправляет ход серверу и отображает результат (быки и коровы)
10. Отслеживает завершение игры и возвращается в меню по окончании

game_logic.c - Игровая логика:

1. Содержит функции генерации секретного числа (4 уникальные цифры)
2. Реализует валидацию ходов игроков (проверка формата и уникальности цифр)
3. Считает быков (правильные цифры на своих местах) и коров (правильные цифры на чужих местах)
4. Управляет состоянием сервера: создание игр, присоединение игроков, поиск подходящих игр
5. Обрабатывает ходы игроков с проверкой очередности и обновлением состояния игры
6. Реализует алгоритм поиска "лучшей" игры для автоматического присоединения
7. Обеспечивает функции отображения списка игр и состояния игры

game_logic.h - Заголовочный файл игровой логики:

1. Объявляет структуры данных: игрок, ход, состояние игры, сервер, запрос, ответ
2. Определяет константы: максимальное количество игр, игроков, ходов, размеры имен
3. Объявляет функции для работы с игровой логикой и обработки запросов

os.h - Кроссплатформенный интерфейс:

1. Определяет структуры данных для игры: Player, GameMove, GameState, GameServer, ClientRequest, ServerResponse, CommunicationChannel
2. Объявляет константы для имен разделяемой памяти и синхронизационных объектов
3. Задаёт типы запросов: создание игры, присоединение, поиск, ход, состояние
4. Объявляет функции для работы с разделяемой памятью, мьютексом, семафорами
5. Обеспечивает переносимость между Windows и Linux системами (в текущей реализации только Linux)

os_linux.c - Linux-реализация системных функций:

1. Реализует все функции из os.h для Linux-систем
2. Создает и управляет разделяемой памятью для межпроцессного взаимодействия
3. Реализует именованные мьютексы и семафоры для синхронизации между процессами
4. Обеспечивает работу с процессами и разделяемой памятью в Linux-системах

Используемые системные вызовы:

1. shm_open() - создание/открытие объектов разделяемой памяти для сервера и канала запросов, а также для именованного мьютекса
2. ftruncate() - установка размера объектов разделяемой памяти под структуры GameServer, CommunicationChannel и pthread_mutex_t
3. mmap() - отображение разделяемой памяти в адресное пространство процессов сервера и клиентов
4. munmap() - удаление отображения разделяемой памяти при завершении процессов
5. shm_unlink() - удаление объектов разделяемой памяти при завершении сервера
6. sem_open() - создание/открытие именованных семафоров для синхронизации
7. sem_wait() - блокирующее ожидание на семафоре
8. sem_post() - разблокировка семафора
9. sem_unlink() - удаление именованных семафоров при завершении сервера
10. pthread_mutex_init() - инициализация именованного мьютекса для синхронизации доступа к разделяемой памяти
11. pthread_mutex_lock() - блокировка мьютекса перед доступом к критическим секциям
12. pthread_mutex_unlock() - разблокировка мьютекса после завершения работы с критическими секциями
13. pthread_mutex_destroy() - уничтожение мьютекса при завершении работы
14. getpid() - получение PID процессов для идентификации клиентов и сервера
15. signal() - установка обработчиков сигналов для корректного завершения сервера
16. close() - закрытие файловых дескрипторов разделяемой памяти
17. getchar()/fgets() - чтение пользовательского ввода в клиентском процессе
18. printf()/fprintf() - вывод информации в консоль для пользователей и отладки

Результаты

При создании игры указывается количество игроков (2-4 человека). Игра активируется при полном наборе игроков. Игроки ходят по очереди, угадывая 4-значное число без повторяющихся цифр. Сервер подсчитывает "быков"(правильные цифры на своих местах) и "коров"(правильные цифры на чужих местах).

Пример работы:

```
./server
=== сервер ===
сервер запущен (PID: 5138)
создана разделяемая память:
- данные сервера: /bulls_cows_server
- запросы: /bulls_cows_requests
- мьютекс: /bulls_cows_mutex
- семафоры: /bulls_cows_sem_req, /bulls_cows_sem_resp
```

нажмите Ctrl+C, чтобы остановить сервер

...

(id=1, index=0)

Server DEBUG: make_move called for game 'g1'

Server DEBUG: player_id=1, guess=' 5841'

Server DEBUG: current_player_turn=0, player_count=2

Server DEBUG: Correct player's turn: polina (id=1)

сервер: поворот изменился с 'polina' (id=1, index=0) на 'anna' (id=2, index=1)

Server DEBUG: make_move called for game 'g1'

Server DEBUG: player_id=2, guess=' 5861'

Server DEBUG: current player turn=1, player count=2

Server DEBUG: Correct player's turn: anna (id=2)

сервер: игра 'g1' завершена! победитель: анна

сервер: удаление старой игры "g1"

./client

=== клиент ===

PID: 11114

введите имя: polina

подключено к серверу

=== быки и коровы ===

1. создать новую игру
2. присоединиться к игре по имени
3. найти игру
4. показать все игры
5. выход

выберите: 1

введите название игры: g1

максимум игроков (2-4): 2

игра 'g1' создана

ваш ID: 1

ожидание 1 игроков

=== игра: g1 ===

игроки: 1/2 | статус: ожидание

ожидание игроков...

игроков: 1/2

...

./client

=== клиент ===

PID: 11198

введите имя: анна

подключено к серверу

=== быки и коровы ===

1. создать новую игру
2. присоединиться к игре по имени
3. найти игру
4. показать все игры

5. выход
выберите: 2
=== доступные игры (1) ===
1. 'g1' (1/2 игроки) - ожидание игроков
введите название игры: g1
...
=== игра: g1 ===
игроки: 2/2 | статус: активна | ход: аппа
последние ходы:
игрок 1: 5891 (быки: 3, коровы: 0)
игрок 2: 5831 (быки: 3, коровы: 0)
игрок 1: 5801 (быки: 3, коровы: 0)
игрок 2: 5831 (быки: 3, коровы: 0)
игрок 1: 5841 (быки: 3, коровы: 0)
»> ВАШ ХОД «<
введите 4 цифр: 5861
ваш ход: 5861 (быки: 4, коровы: 0)
ПОБЕДА! вы угадали!
нажмите Enter

Выводы

Разработан программный прототип консольно-серверной игры "Быки и коровы" с архитектурой "клиент-сервер". Сервер выступает центральным координатором, обеспечивающим взаимодействие между клиентами. Клиенты являются независимыми процессами игроков, подключающимися к серверу.

Межпроцессное взаимодействие организовано через разделяемую память с использованием именованных семафоров для синхронизации. Такой подход обеспечивает эффективную передачу данных между процессами.

Игровая механика поддерживает от 2 до 4 игроков в одной сессии. При создании игры указывается её имя и максимальное число участников. Сервер автоматически активирует игру, когда набирается нужное количество игроков, после чего участники по очереди пытаются угадать секретное четырёхзначное число.

Архитектурно система разделена на два независимых процесса — сервер и клиент. Сервер работает в фоновом режиме, обрабатывает запросы, обновляет состояние игр и удаляет неактивные сессии.

Именованные мьютексы гарантируют корректный доступ к общим данным при параллельной работе множества клиентов, предотвращая состояния гонки и повреждение данных. Семафорная сигнализация позволяет избежать активного ожидания, снижая нагрузку на процессор.

Таким образом, разработанная система демонстрирует работоспособный подход к организации многопользовательских игр через разделяемую память и может служить основой для более сложных распределённых приложений.

Исходная программа

```
1  #pragma once
2  #define OS_H
3
4  #ifdef _WIN32
5  #include <windows.h>
6  #else
7  #include <fcntl.h>
8  #include <stdio.h>
9  #include <stdlib.h>
10 #include <string.h>
11 #include <unistd.h>
12 #include <sys/mman.h>
13 #include <sys/stat.h>
14 #include <pthread.h>
15 #include <time.h>
16 #include <semaphore.h>
17 #endif
18
19
20 #define MAX_GAMES 10
21 #define MAX_PLAYERS 4
22 #define MAX_MOVES 100
23 #define MAX_GAME_NAME 50
24 #define MAX_PLAYER_NAME 50
25 #define MAX_REQUESTS 20
26 #define SECRET_LENGTH 4
27
28 #define SHM_SERVER "/bulls_cows_server"
29 #define SHM_REQUESTS "/bulls_cows_requests"
30 #define MUTEX_SERVER "/bulls_cows_mutex"
31 #define SEM_REQUEST "/bulls_cows_sem_req"
32 #define SEM_RESPONSE "/bulls_cows_sem_resp"
33
34 #define REQ_CREATE 1
35 #define REQ_JOIN 2
36 #define REQ_FIND 3
37 #define REQ_MAKE_MOVE 4
38 #define REQ_GET_STATE 5
39
40 typedef struct {
41     int id;
42     char name[MAX_PLAYER_NAME];
43     int connected;
44     int score;
45     time_t last_activity;
46 } Player;
47
48 typedef struct {
49     int player_id;
50     char guess[SECRET_LENGTH + 1];
51     int bulls;
52     int cows;
53     time_t timestamp;
54 } GameMove;
55
56 typedef struct {
```

```

57     char game_name[MAX_GAME_NAME];
58     char secret_number[SECRET_LENGTH + 1];
59     int player_count;
60     int max_players;
61     int current_player_turn;
62     Player players[MAX_PLAYERS];
63     GameMove moves[MAX_MOVES];
64     int move_count;
65     int is_active;
66     int is_finished;
67     char winner[MAX_PLAYER_NAME];
68     time_t created_time;
69 } GameState;
70
71 //
72 typedef struct {
73     int server_pid;
74     int game_count;
75     GameState games[MAX_GAMES];
76     int server_running;
77 } GameServer;
78
79 //
80 typedef struct {
81     int type; //
82     int client_pid; //PID
83     int request_id; //ID
84
85     union {
86         struct {
87             char game_name[MAX_GAME_NAME];
88             char player_name[MAX_PLAYER_NAME];
89             int max_players;
90         } create;
91
92         struct {
93             char game_name[MAX_GAME_NAME];
94             char player_name[MAX_PLAYER_NAME];
95         } join;
96
97         struct {
98             char player_name[MAX_PLAYER_NAME];
99         } find;
100
101         struct {
102             int game_index;
103             int player_id;
104             char guess[SECRET_LENGTH + 1];
105         } move;
106
107         struct {
108             int game_index;
109             int player_id;
110         } state;
111     } data;
112
113     time_t timestamp;
114 } ClientRequest;

```



```

115
116 //
117 typedef struct {
118     int request_id; //ID
119     int client_pid; //PID
120     int success; //1 - , 0 -
121
122     union {
123         struct {
124             int game_index;
125             int player_id;
126         } game_info;
127
128         struct {
129             int bulls;
130             int cows;
131         } move_result;
132
133         struct {
134             char game_name[MAX_GAME_NAME];
135             int player_count;
136             int max_players;
137             int is_active;
138             int is_finished;
139             int current_player_turn;
140             char current_player[MAX_PLAYER_NAME];
141             GameMove last_moves[5];
142             int last_moves_count;
143         } game_state;
144     } data;
145
146     char error_msg[100];
147 } ServerResponse;
148
149 //
150 typedef struct {
151     ClientRequest requests[MAX_REQUESTS];
152     ServerResponse responses[MAX_REQUESTS];
153     int request_count;
154     int response_count;
155 } CommunicationChannel;
156
157 //shared memory
158 int create_shm(const char* name, size_t size);
159 int open_shm(const char* name);
160 void* map_shm(int fd, size_t size);
161 int unmap_shm(void* addr, size_t size);
162 int close_shm(int fd);
163 int unlink_shm(const char* name);
164
165 //
166 int create_mutex(void);
167 int open_mutex(void);
168 int lock_mutex(void);
169 int unlock_mutex(void);
170 int close_mutex(void);
171
172 //

```

```

173 int create_semaphore(const char* name, int initial);
174 int open_semaphore(const char* name);
175 int wait_semaphore(const char* name);
176 int post_semaphore(const char* name);
177 int close_semaphore(const char* name);
178 int unlink_semaphore(const char* name);
179
180 void sleep_ms(int ms);
181 time_t get_time(void);
182 int get_pid(void);

```

Листинг 1: os.h

```

1 #pragma once
2 #define GAME_LOGIC_H
3
4 #include "os.h"
5
6 void generate_secret(char* secret);
7 int is_valid_guess(const char* guess);
8 void calculate_bulls_cows(const char* secret, const char* guess, int* bulls, int* cows
   );
9
10 void init_server(GameServer* server);
11 int server_create_game(GameServer* server, const char* name, const char* player_name,
12     int max_players, int* game_idx, int* player_id);
13 int server_join_game(GameServer* server, const char* game_name,
14     const char* player_name, int* game_idx, int* player_id);
15 int server_find_game(GameServer* server, const char* player_name,
16     int* game_idx, int* player_id);
17 int server_make_move(GameServer* server, int game_idx, int player_id,
18     const char* guess, int* bulls, int* cows);
19 void server_get_game_state(GameServer* server, int game_idx, int player_id,
20     ServerResponse* response);
21
22 int add_client_request(CommunicationChannel* channel, const ClientRequest* request);
23 int get_client_response(CommunicationChannel* channel, int client_pid,
24     int request_id, ServerResponse* response);
25 void process_client_requests(GameServer* server, CommunicationChannel* channel);
26
27 void check_inactive_players(GameServer* server);
28 void remove_old_games(GameServer* server);
29 void activate_games(GameServer* server);
30
31 int client_send_request(CommunicationChannel* channel, const ClientRequest* request);
32 int client_wait_response(CommunicationChannel* channel, int client_pid,
33     int request_id, ServerResponse* response, int timeout_ms);
34
35 void print_game_list(GameServer* server);
36 void print_game_state_from_response(ServerResponse* response);
37 void print_menu(void);
38 void print_error(const char* msg);

```

Листинг 2: game_logic.h

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <signal.h>
4  #include <unistd.h>
5
6  #include "game_logic.h"
7
8  static volatile int running = 1;
9
10 void handle_signal(int sig) {
11     running = 0;
12     printf("\n:   %d,   \n", sig);
13 }
14
15 int main() {
16     printf("===   ===\n");
17
18     //
19     signal(SIGINT, handle_signal);
20     signal(SIGTERM, handle_signal);
21
22     // shared memory
23     int shm_server_fd = create_shm(SHM_SERVER, sizeof(GameServer));
24     if (shm_server_fd == -1) {
25         fprintf(stderr, "Failed to create server shared memory\n");
26         return 1;
27     }
28
29     // shared memory
30     int shm_requests_fd = create_shm(SHM_REQUESTS, sizeof(CommunicationChannel));
31     if (shm_requests_fd == -1) {
32         fprintf(stderr, "Failed to create requests shared memory\n");
33         close_shm(shm_server_fd);
34         unlink_shm(SHM_SERVER);
35         return 1;
36     }
37
38     GameServer* server = map_shm(shm_server_fd, sizeof(GameServer));
39     CommunicationChannel* channel = map_shm(shm_requests_fd, sizeof(
        CommunicationChannel));
40
41     if (!server || !channel) {
42         fprintf(stderr, "Failed to map shared memory\n");
43         if (server) unmap_shm(server, sizeof(GameServer));
44         if (channel) unmap_shm(channel, sizeof(CommunicationChannel));
45         close_shm(shm_server_fd);
46         close_shm(shm_requests_fd);
47         unlink_shm(SHM_SERVER);
48         unlink_shm(SHM_REQUESTS);
49         return 1;
50     }
51
52     //
53     memset(channel, 0, sizeof(CommunicationChannel));
54
55     if (create_mutex() == -1) {
56         fprintf(stderr, "Failed to create mutex\n");
57         unmap_shm(server, sizeof(GameServer));

```

```

58     unmap_shm(channel, sizeof(CommunicationChannel));
59     close_shm(shm_server_fd);
60     close_shm(shm_requests_fd);
61     unlink_shm(SHM_SERVER);
62     unlink_shm(SHM_REQUESTS);
63     return 1;
64 }
65
66 if (create_semaphore(SEM_REQUEST, 0) == -1 ||
67     create_semaphore(SEM_RESPONSE, 0) == -1) {
68     fprintf(stderr, "Failed to create semaphores\n");
69     close_mutex();
70     unmap_shm(server, sizeof(GameServer));
71     unmap_shm(channel, sizeof(CommunicationChannel));
72     close_shm(shm_server_fd);
73     close_shm(shm_requests_fd);
74     unlink_shm(SHM_SERVER);
75     unlink_shm(SHM_REQUESTS);
76     return 1;
77 }
78
79 //
80 lock_mutex();
81 init_server(server);
82 unlock_mutex();
83
84 printf(" (PID: %d)\n", get_pid());
85 printf(" :\n");
86 printf(" - : %s\n", SHM_SERVER);
87 printf(" - : %s\n", SHM_REQUESTS);
88 printf(" - : %s\n", MUTEX_SERVER);
89 printf(" - : %s, %s\n", SEM_REQUEST, SEM_RESPONSE);
90 printf("\n Ctrl+C,  \n");
91
92 while (running && server->server_running) {
93     if (wait_semaphore(SEM_REQUEST) == 0) {
94         lock_mutex();
95         process_client_requests(server, channel);
96
97         if (channel->response_count > 0) {
98             post_semaphore(SEM_RESPONSE);
99         }
100
101         unlock_mutex();
102     }
103
104     lock_mutex();
105
106     activate_games(server);
107
108     check_inactive_players(server);
109
110     remove_old_games(server);
111
112     unlock_mutex();
113
114     sleep_ms(50);
115 }

```

```

116
117     printf("\n:  \n");
118
119     lock_mutex();
120     server->server_running = 0;
121     unlock_mutex();
122
123     close_mutex();
124     unlink_semaphore(SEM_REQUEST);
125     unlink_semaphore(SEM_RESPONSE);
126
127     unmap_shm(server, sizeof(GameServer));
128     unmap_shm(channel, sizeof(CommunicationChannel));
129
130     close_shm(shm_server_fd);
131     close_shm(shm_requests_fd);
132
133     unlink_shm(SHM_SERVER);
134     unlink_shm(SHM_REQUESTS);
135
136     printf("\n");
137
138     return 0;
139 }

```

Листинг 3: server.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include "game_logic.h"
5
6  static int client_pid = 0;
7  static int request_counter = 0;
8
9  void clear_input_buffer() {
10     int c;
11     while ((c = getchar()) != '\n' && c != EOF);
12 }
13
14 int get_request_id() {
15     return ++request_counter;
16 }
17
18 int main() {
19     static int last_shown_turn_index = -1;
20     static int consecutive_wait_count = 0;
21
22     client_pid = get_pid();
23
24     printf("===  ===\n");
25     printf("PID: %d\n", client_pid);
26
27     char player_name[MAX_PLAYER_NAME];
28     printf(" : ");
29
30     if (!fgets(player_name, sizeof(player_name), stdin)) {

```

```

31     printf("    \n");
32     return 1;
33 }
34 player_name[strcspn(player_name, "\n")] = '\0';
35
36 if (strlen(player_name) == 0) {
37     printf("\n");
38     return 1;
39 }
40
41 // shared memory
42 int shm_server_fd = open_shm(SHM_SERVER);
43 if (shm_server_fd == -1) {
44     printf(":    \n");
45     return 1;
46 }
47
48 // shared memory
49 int shm_requests_fd = open_shm(SHM_REQUESTS);
50 if (shm_requests_fd == -1) {
51     printf(":    \n");
52     close_shm(shm_server_fd);
53     return 1;
54 }
55
56 GameServer* server = map_shm(shm_server_fd, sizeof(GameServer));
57 CommunicationChannel* channel = map_shm(shm_requests_fd, sizeof(
    CommunicationChannel));
58
59 if (!server || !channel) {
60     printf(":    \n");
61     if (server) unmap_shm(server, sizeof(GameServer));
62     if (channel) unmap_shm(channel, sizeof(CommunicationChannel));
63     close_shm(shm_server_fd);
64     close_shm(shm_requests_fd);
65     return 1;
66 }
67
68 if (open_mutex() == -1) {
69     printf(":    \n");
70     unmap_shm(server, sizeof(GameServer));
71     unmap_shm(channel, sizeof(CommunicationChannel));
72     close_shm(shm_server_fd);
73     close_shm(shm_requests_fd);
74     return 1;
75 }
76
77 if (open_semaphore(SEM_REQUEST) == -1 ||
78     open_semaphore(SEM_RESPONSE) == -1) {
79     printf(":    \n");
80     close_mutex();
81     unmap_shm(server, sizeof(GameServer));
82     unmap_shm(channel, sizeof(CommunicationChannel));
83     close_shm(shm_server_fd);
84     close_shm(shm_requests_fd);
85     return 1;
86 }
87

```



```

146         req.timestamp = get_time();
147
148         strcpy(req.data.create.game_name, game_name);
149         strcpy(req.data.create.player_name, player_name);
150         req.data.create.max_players = max_players;
151
152         if (client_send_request(channel, &req) == -1) {
153             printf("    \n");
154             break;
155         }
156
157         ServerResponse resp;
158         if (client_wait_response(channel, client_pid, req.request_id,
159                                 &resp, 5000) == -1) {
160             printf("    \n");
161             break;
162         }
163
164         if (resp.success) {
165             current_game_index = resp.data.game_info.game_index;
166             current_player_id = resp.data.game_info.player_id;
167             strcpy(current_game_name, game_name);
168             in_game = 1;
169
170             //
171             last_move_count = -1;
172             last_player_turn = -1;
173             last_is_finished = -1;
174             last_is_active = -1;
175             debug_shown = 0;
176
177             printf(" '%s' \n", game_name);
178             printf(" ID: %d\n", current_player_id);
179             printf(" %d \n", max_players - 1);
180         } else {
181             printf("    : %s\n", resp.error_msg);
182         }
183         break;
184     }
185
186     case 2: {
187         //
188         lock_mutex();
189         print_game_list(server);
190         unlock_mutex();
191
192         char game_name[MAX_GAME_NAME];
193         printf("\n    : ");
194         fgets(game_name, sizeof(game_name), stdin);
195         game_name[strcspn(game_name, "\n")] = '\0';
196
197         if (strlen(game_name) == 0) {
198             printf("    \n");
199             break;
200         }
201
202         ClientRequest req;
203         memset(&req, 0, sizeof(ClientRequest));

```



```

204     req.type = REQ_JOIN;
205     req.client_pid = client_pid;
206     req.request_id = get_request_id();
207     req.timestamp = get_time();
208
209     strcpy(req.data.join.game_name, game_name);
210     strcpy(req.data.join.player_name, player_name);
211
212     if (client_send_request(channel, &req) == -1) {
213         printf("    \n");
214         break;
215     }
216
217     ServerResponse resp;
218     if (client_wait_response(channel, client_pid, req.request_id,
219                             &resp, 5000) == -1) {
220         printf("    \n");
221         break;
222     }
223
224     if (resp.success) {
225         current_game_index = resp.data.game_info.game_index;
226         current_player_id = resp.data.game_info.player_id;
227
228         //
229         lock_mutex();
230         if (current_game_index >= 0 &&
231             current_game_index < server->game_count) {
232             strcpy(current_game_name,
233                   server->games[current_game_index].game_name);
234         }
235         unlock_mutex();
236
237         in_game = 1;
238
239         //
240         last_move_count = -1;
241         last_player_turn = -1;
242         last_is_finished = -1;
243         last_is_active = -1;
244         debug_shown = 0;
245
246         printf("    '%s'\n", current_game_name);
247         printf("    ID: %d\n", current_player_id);
248     } else {
249         printf("    : %s\n", resp.error_msg);
250     }
251     break;
252 }
253
254 case 3: {
255     printf("    ...\n");
256
257     ClientRequest req;
258     memset(&req, 0, sizeof(ClientRequest));
259     req.type = REQ_FIND;
260     req.client_pid = client_pid;
261     req.request_id = get_request_id();

```

```

262         req.timestamp = get_time();
263
264         strcpy(req.data.find.player_name, player_name);
265
266         if (client_send_request(channel, &req) == -1) {
267             printf("    \n");
268             break;
269         }
270
271         ServerResponse resp;
272         if (client_wait_response(channel, client_pid, req.request_id,
273             &resp, 5000) == -1) {
274             printf("    \n");
275             break;
276         }
277
278         if (resp.success) {
279             current_game_index = resp.data.game_info.game_index;
280             current_player_id = resp.data.game_info.player_id;
281
282             lock_mutex();
283             if (current_game_index >= 0 &&
284                 current_game_index < server->game_count) {
285                 strcpy(current_game_name,
286                     server->games[current_game_index].game_name);
287             }
288             unlock_mutex();
289
290             in_game = 1;
291
292             //
293             last_move_count = -1;
294             last_player_turn = -1;
295             last_is_finished = -1;
296             last_is_active = -1;
297             debug_shown = 0;
298
299             printf("    '%s'\n", current_game_name);
300             printf("    ID: %d\n", current_player_id);
301         } else {
302             printf("    : %s\n", resp.error_msg);
303             printf("    \n");
304         }
305         break;
306     }
307
308     case 4: {
309         //
310         lock_mutex();
311         print_game_list(server);
312         unlock_mutex();
313         break;
314     }
315
316     case 5: {
317         //
318         printf("\n");
319

```

```

320         close_mutex();
321         unmap_shm(server, sizeof(GameServer));
322         unmap_shm(channel, sizeof(CommunicationChannel));
323         close_shm(shm_server_fd);
324         close_shm(shm_requests_fd);
325
326         return 0;
327     }
328
329     default:
330         printf(" \n");
331     }
332
333 } else {
334     //
335     ClientRequest state_req;
336     memset(&state_req, 0, sizeof(ClientRequest));
337     state_req.type = REQ_GET_STATE;
338     state_req.client_pid = client_pid;
339     state_req.request_id = get_request_id();
340     state_req.timestamp = get_time();
341     state_req.data.state.game_index = current_game_index;
342     state_req.data.state.player_id = current_player_id;
343
344     if (client_send_request(channel, &state_req) == -1) {
345         printf(" \n");
346         sleep_ms(2000);
347         continue;
348     }
349
350     ServerResponse state_resp;
351     if (client_wait_response(channel, client_pid, state_req.request_id,
352                             &state_resp, 3000) == -1) {
353         printf("-\n");
354         sleep_ms(2000);
355         continue;
356     }
357
358     if (!state_resp.success) {
359         printf(": %s\n", state_resp.error_msg);
360         sleep_ms(2000);
361         continue;
362     }
363
364     // ,
365     int state_changed = 0;
366
367     if (last_move_count != state_resp.data.game_state.last_moves_count ||
368         last_player_turn != state_resp.data.game_state.current_player_turn ||
369         last_is_finished != state_resp.data.game_state.is_finished ||
370         last_is_active != state_resp.data.game_state.is_active) {
371
372         state_changed = 1;
373
374         //
375         last_move_count = state_resp.data.game_state.last_moves_count;
376         last_player_turn = state_resp.data.game_state.current_player_turn;
377         last_is_finished = state_resp.data.game_state.is_finished;

```

```

378         last_is_active = state_resp.data.game_state.is_active;
379
380         //
381         printf("\n=== : %s ===\n", state_resp.data.game_state.game_name);
382         printf(": %d/%d | : ",
383             state_resp.data.game_state.player_count,
384             state_resp.data.game_state.max_players);
385
386         if (state_resp.data.game_state.is_finished) {
387             printf("\n");
388         } else if (state_resp.data.game_state.is_active) {
389             printf(" | : %s\n",
390                 state_resp.data.game_state.current_player);
391         } else {
392             printf("\n");
393         }
394     }
395
396     //
397     if (state_resp.data.game_state.is_finished) {
398         if (state_changed) {
399             printf("\n===   ===\n");
400             printf(" Enter   ");
401             getchar();
402         } else {
403             sleep_ms(1000);
404         }
405         continue;
406     }
407
408     //
409     if (!state_resp.data.game_state.is_active) {
410         if (state_changed) {
411             printf(" ... \n");
412             printf(": %d/%d\n",
413                 state_resp.data.game_state.player_count,
414                 state_resp.data.game_state.max_players);
415         }
416         sleep_ms(3000);
417         continue;
418     }
419
420     // ,
421     int is_my_turn = 0;
422
423     //
424     if (state_changed || !debug_shown) {
425         debug_shown = 1;
426
427         //
428         if (state_resp.data.game_state.last_moves_count > 0) {
429             printf("\n : \n");
430             for (int i = 0; i < state_resp.data.game_state.last_moves_count; i
431                 ++){
432                 GameMove* move = &state_resp.data.game_state.last_moves[i];
433                 printf(" %d: %s (: %d, : %d)\n",
434                     move->player_id, move->guess, move->bulls, move->cows);
435             }

```

```

435     }
436
437     //
438     if (strcmp(state_resp.data.game_state.current_player, player_name) ==
439         0) {
440         is_my_turn = 1;
441         printf("\n>>> <<<\n");
442     } else {
443         printf("\n>>> : %s <<<\n",
444             state_resp.data.game_state.current_player);
445     }
446     //
447     if (strcmp(state_resp.data.game_state.current_player, player_name) ==
448         0) {
449         is_my_turn = 1;
450     }
451
452     //
453     if (state_changed) {
454         debug_shown = 0;
455     }
456
457     if (is_my_turn) {
458         //
459         char guess[10];
460         printf(" %d : ", SECRET_LENGTH);
461         fgets(guess, sizeof(guess), stdin);
462         guess[strcspn(guess, "\n")] = '\0';
463
464         if (!is_valid_guess(guess)) {
465             printf("! %d (0-9)\n", SECRET_LENGTH);
466             sleep_ms(1000);
467             continue;
468         }
469
470         //
471         ClientRequest move_req;
472         memset(&move_req, 0, sizeof(ClientRequest));
473         move_req.type = REQ_MAKE_MOVE;
474         move_req.client_pid = client_pid;
475         move_req.request_id = get_request_id();
476         move_req.timestamp = get_time();
477         move_req.data.move.game_index = current_game_index;
478         move_req.data.move.player_id = current_player_id;
479         strcpy(move_req.data.move.guess, guess);
480
481         if (client_send_request(channel, &move_req) == -1) {
482             printf(" \n");
483             sleep_ms(1000);
484             continue;
485         }
486
487         ServerResponse move_resp;
488         if (client_wait_response(channel, client_pid, move_req.request_id,
489             &move_resp, 3000) == -1) {
490             printf("-\n");

```

```

491         sleep_ms(1000);
492         continue;
493     }
494
495     if (move_resp.success) {
496         printf(" : %s (: %d, : %d)\n",
497             guess, move_resp.data.move_result.bulls,
498             move_resp.data.move_result.cows);
499
500         if (move_resp.data.move_result.bulls == SECRET_LENGTH) {
501             printf("\n! !\n");
502             printf(" Enter\n");
503             getchar();
504         }
505     } else {
506         printf(": %s\n", move_resp.error_msg);
507     }
508
509     } else {
510         // -
511         printf(".");
512         fflush(stdout);
513         sleep_ms(2500);
514     }
515 }
516
517 return 0;
518 }

```

Листинг 4: client.c